

Web Application Penetration Testing Using Samurai

This article introduces you to penetration-testing of Web applications. Also included is some hands-on work with a Web-testing framework called Samurai.

Penetration testing, affectionately called pen-testing, involves testing a piece of software, or network and security systems, for resilience against attacks from crackers (or black hat hackers—the bad guys). Nothing is perfect; we all know that. Every application, website or network has vulnerabilities. Pen-testing allows us to detect those holes so that we can fix them and prevent exploits. It's a very geeky domain, so I will keep things simple here.

Penetration testing is conceptually divided into:

- Black-box testing: As the name implies, the internals are not taken into consideration for this type of testing. How the software actually works (or the network functions) is not relevant here. The functionality of the software or network is considered, and then the testing is carried out.
- White-box testing: The opposite of black-box testing, this involves the internal workings of the system; the workings of the software or network are probed and tested for holes.

Web application pen-testing

Since the domain of penetration testing is a broad one, this article will focus on pen-testing Web applications. To aid understanding, we will be doing some hands-on work with a neat framework called Samurai. There are many

methodologies for testing Web applications. There is a comprehensive one by OWASP (the Open Web Application Security Project). This lists a methodology with nine sub-categories: configuration management testing, business logic testing, authentication testing, session management testing, authorisation testing, data validation testing, denial of service testing, Web services testing and AJAX testing. To keep things simple, I will explain a widely used 4-stage methodology (used by Samurai too) for testing Web applications.

Stage 1: Reconnaissance

The first stage in pen-testing a Web application is to find what information about the application is already available out there. What servers does it run on? What back-end technology does it use? What environment does it run in, apart from details like its infrastructure, its hosts, machines, operating systems, network details, etc? Basically, this is an information-gathering stage. Getting to know our target is what is going on at this stage.

Stage 2: Mapping

What are the bits and pieces that make up the application? What makes it tick? What are the relationships between